

Technische Dokumentation: Retail Data Pipeline & BI-Lösung

Architektur: Von normalisierten OLTP-Strukturen zum analytischen Sternschema

1. Architektur-Überblick

Das primäre Ziel dieses Projekts war die Entwicklung einer skalierbaren Dateninfrastruktur, um transaktionale Rohdaten in entscheidungsrelevante analytische Strukturen zu transformieren. Die Pipeline ist in drei Ebenen gegliedert:

- **Datenquelle:** Ingestion von transaktionalen Datensätzen (über 500.000 Datensätze).
- **Operative Ebene (Retail_OLTP):** Ein relationales Modell in der 3. Normalform (3NF), konzipiert für ACID-Konformität und maximale Datenintegrität.
- **Analytische Ebene (Retail_DWH):** Ein denormalisiertes Sternschema (Star Schema), optimiert für OLAP-Workloads und komplexes Business-Reporting.

2. Datenmodellierung & Design

A) Operative Datenbank (OLTP)

Das Schema wurde entwickelt, um Redundanzen zu minimieren und die referenzielle Integrität zu gewährleisten:

- **Stammdatenverwaltung:** Zentralisierte Entitäten für Filialen (Stores), Mitarbeiter und Kunden.
- **Transaktionsmodellierung:** Implementierung eines Master-Detail-Musters (SalesHeaders/SalesDetails), um Rechnungsmetadaten von der Granularität der einzelnen Positionen zu entkoppeln.

B) Data Warehouse (DWH)

Die Daten wurden in ein multidimensionales Sternschema überführt, um die Join-Komplexität zu reduzieren:

- **Faktentabelle (Sales_Fact):** Konsolidierung von Metriken (Menge, Einzelpreis, Gesamtpreis) unter Verwendung von Surrogat-Schlüsseln zur Performance-Steigerung.
- **Dimensionen:** Fünf Analyse-Achsen (Zeit, Produkt, Kunde, Filiale, Mitarbeiter) für ein präzises "Slicing and Dicing" der Retail-KPIs.

3. Technische Herausforderungen & Engineering-Lösungen

Herausforderung 1: Heuristische Datenkategorisierung

- **Problem:** Fehlende Produktkategorien in den Quelldaten.
- **Lösung:** Entwicklung einer Python-basierten Mapping-Engine. Mittels Regex und Keyword-Matching in der Spalte Description wurden Produkte programmatisch logischen Geschäftskategorien zugewiesen.

Herausforderung 2: ODBC-Treiber & Datentyp-Kompatibilität

- **Problem:** Auftreten von HYC00-Fehlern aufgrund von Inkompatibilitäten zwischen Python-Datetime und SQL-Server-Formaten beim Bulk-Loading.
- **Lösung:** Standardisierung der ETL-Pipeline durch ISO-konforme String-Konvertierung mittels `.strftime()`. Zusätzlich wurde die Komponente Stunde als separates Feature für die Analyse von Spitzenverkaufszeiten extrahiert.

Herausforderung 4: Historische Datenintegrität (SCD Typ 2)

- **Problem:** Preisfluktuationen im Quellsystem drohten die historischen Umsatzberichte zu verfälschen (rückwirkende Fehler).
- **Lösung:** Implementierung von **Slowly Changing Dimensions (SCD) Typ 2** in der Tabelle `Product_Dim`. Durch die Felder `IsActive`, `Start_Date` und `End_Date` wird ein vollständiger Audit-Trail aller Preisversionen aufrechterhalten.

Herausforderung 4: Optimierung der Abfrage-Performance

- **Problem:** Die Echtzeit-Berechnung von `TotalPrice` über 500.000 Zeilen führte zu hohen Latenzzeiten.
- **Lösung:** Optimierung des Schemas durch **Persisted Computed Columns**. Da das Ergebnis physisch auf der Festplatte gespeichert wird, konnte die Lese-Performance erheblich gesteigert werden.

4. Data Governance & SCD-Strategie

- **SCD Typ 1:** Anwendung bei Filial- und Mitarbeiterdimensionen, bei denen keine historische Nachverfolgung von Metadaten erforderlich war.
- **SCD Typ 2:** Konsequente Anwendung bei Produkt- und Kundendimensionen, um die finanzielle Genauigkeit und langfristige Verhaltensanalysen sicherzustellen.

5. Highlights & Skalierbarkeit

- **Automatisierte Pipeline:** Ein End-to-End-ETL-Workflow unter Verwendung von Python (Pandas & PyODBC) für eine automatisierte Batch-Verarbeitung.
- **BI-Ready:** Das System ist darauf ausgelegt, komplexe Abfragen, wie z. B. regionale Rentabilitätsanalysen nach Stunden, in Sub-Sekundenschnelle zu beantworten.
- **Zukunftssicherheit:** Die Pipeline trennt die Datenbereinigung (Python) vom Ingestion-Prozess (SQL Bulk Inserts), um Datenbank-Locks bei steigendem Datenvolumen zu vermeiden.

Technischer Stack: SQL Server (TSQL), Python 3.x, Pandas, PyODBC.